

Reinforcement Learning

Temporal Difference Learning

Debapriyo Majumdar
Indian Statistical Institute
debapriyo@isical.ac.in

Recall policy evaluation

State-value function

$$v_{\pi}(s) = \mathbb{E}_{\pi} [G_t | S_t = s]$$

$$\text{where } G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} \cdots = R_{t+1} + \gamma G_{t+1}$$

Monte Carlo policy evaluation

Sample G_t repeatedly to estimate $v(S_t)$

$$\text{Update estimate } V(S_t) := V(S_t) + \frac{1}{n+1} [G_t - V(S_t)]$$

at the end of each *episode*

Modification: take constant step-size α

$$\text{Update } V(S_t) := V(S_t) + \alpha [G_t - V(S_t)]$$

Update

$$V(S_t) := V(S_t) + \alpha [R_{t+1} + \gamma \mathbb{E}_{\pi}[G_{t+1} | S_t = s] - V(S_t)]$$

$$V(S_t) := V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

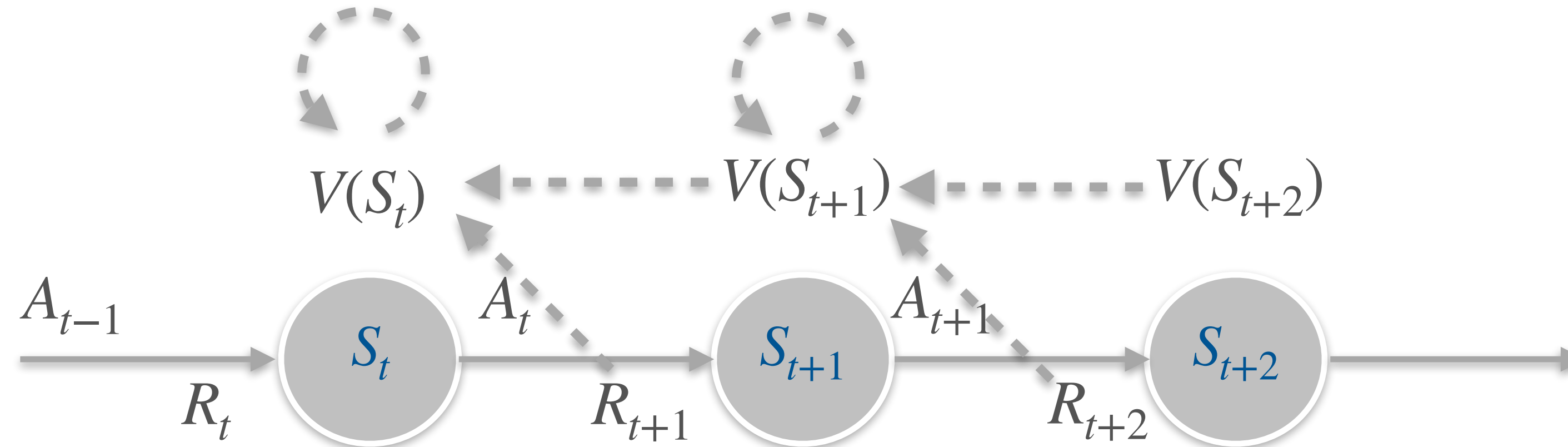
Update $V(S_t)$ as soon as we reach S_{t+1}

Temporal difference learning

Temporal difference (TD) learning

The update formula

$$V(S_t) := V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$



Called 1-step TD, or TD(0)

Example: driving to destination

Going to office, meeting in 1 hour

Reward: amount of time left after arriving (1 hour – time taken), for preparation

For simplicity, we will keep learning estimated time to arrival (ETA)

Assume step-size $\alpha = 1/10$, discount rate $\gamma = 1$

State S	Leaving home	Reach car Raining	Exiting highway	Traffic jam	Entering campus	Arrive
Time elapsed	0	(+5 =) 5	(+15 =) 20	(+10 =) 30	(+10) = 40	(+3 =) 43
ETA based on previous learning	30	35	15	10	3	0
Updated ETA for future, at this state	31m 0s	34m 30s	15m 30s	10m 18s	3m 0s	0

Update ETA(state) := Present ETA(state) + $\frac{1}{10}$ (Time to next state + Present ETA(next state) - Present ETA(state))

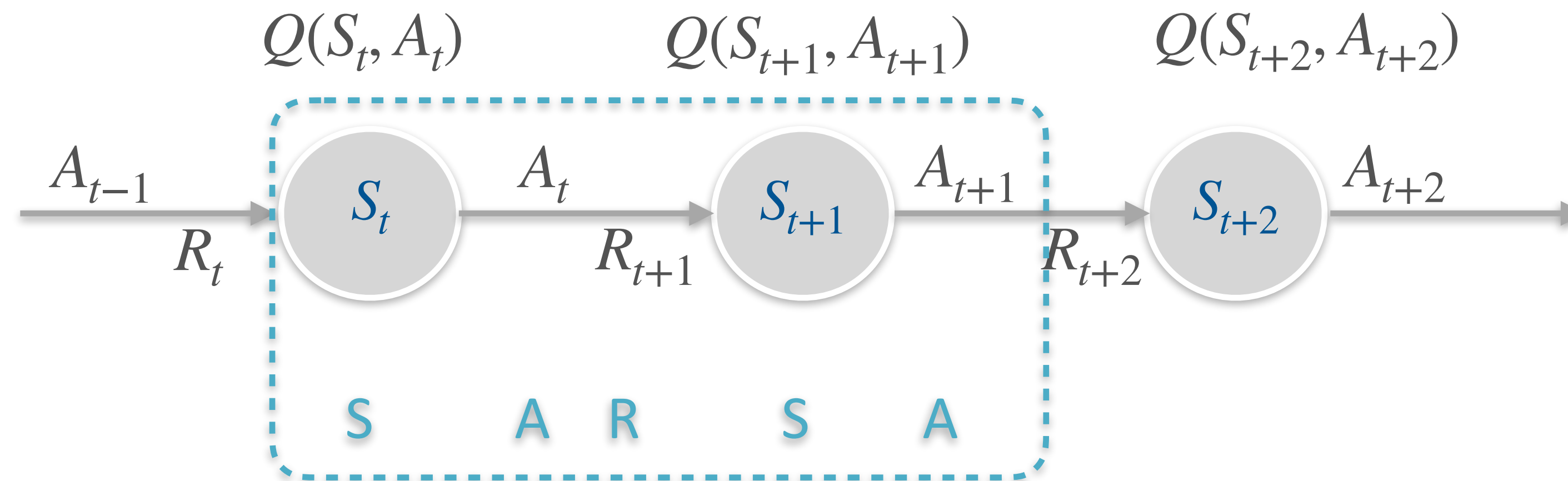
Advantages of TD learning

- Combines the advantages of dynamic programming (DP) and Monte Carlo (MC) methods
 - Similar to DP, a recursive update formula, update estimates based on other estimates
 - Similar to MC, does not require a model, can learn directly from experience
- Key differences with Monte Carlo
 - Monte Carlo can update estimates only after one episode finishes
 - TD updates online and incrementally (compared to MC)
 - Can be applied to tasks with longer episodes or continuing tasks
- TD methods also converge to the correct prediction asymptotically
- But in practice, converges faster than Monte Carlo methods

S A R S A : learning action values

The update formula (analogous to the one for updating state values)

$$Q(S_t, A_t) := Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$



SARSA: On policy TD control

Initialize $Q(s, a) \in \mathbb{R}$ arbitrarily for all s, a pairs, except $Q(S_T, a) = 0$ for terminal S_T and all a

Loop for each episode (an episode can be infinitely long as well)

Initialize start state S_0

Choose action A_0 from S_0 based on policy derived from Q

Can incorporate ε -greedy approach here

For each step t in episode

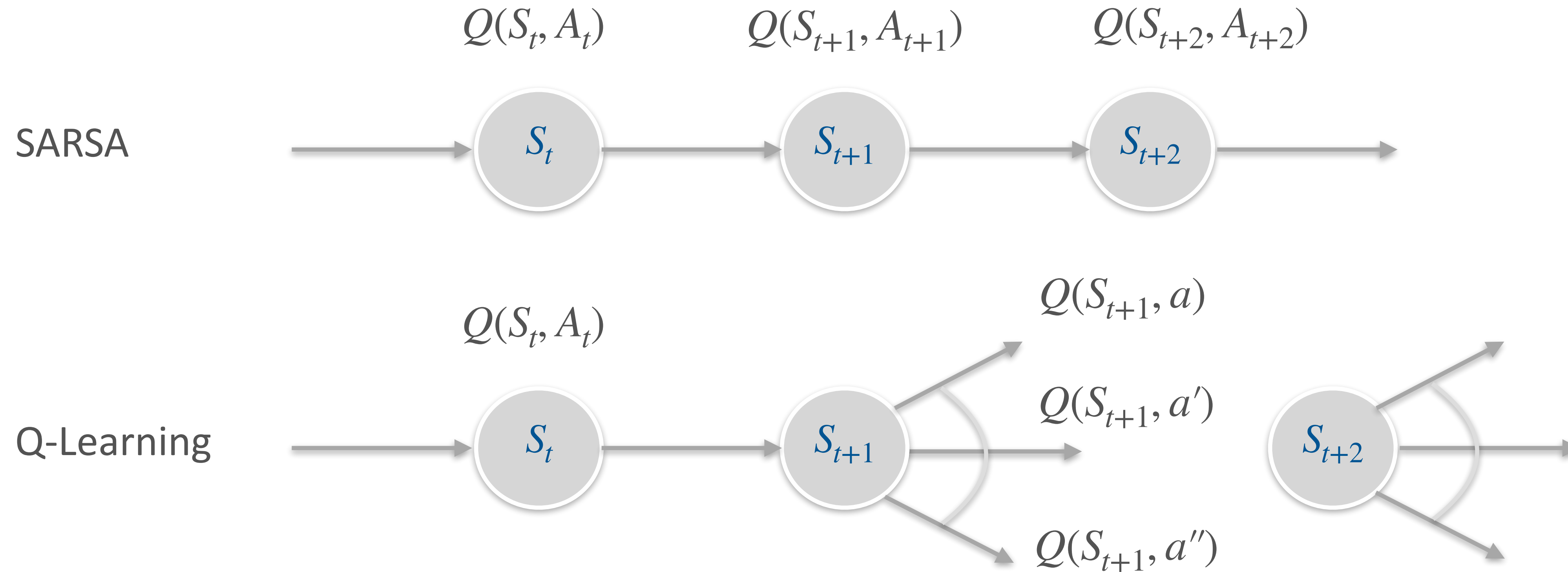
Take action A_t , observe R_{t+1}, S_{t+1}

Choose action A_{t+1} from S_{t+1} based on policy derived from Q

Update $Q(S_t, A_t) := Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$

Until $S_t = S_T$ (terminal state of the episode)

Q-Learning



The update : maximum estimate of possible next actions

$$Q(S_t, A_t) := Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

Q-Learning : off-policy TD control

Initialize $Q(s, a) \in \mathbb{R}$ arbitrarily for all s, a pairs, except $Q(S_T, a) = 0$ for terminal S_T and all a

Loop for each episode (an episode can be infinitely long as well)

Initialize start state S_0

Choose action A_0 from S_0 based on policy derived from Q

Behavior policy

For each step t in episode

Take action A_t , observe R_{t+1}, S_{t+1}

Update $Q(S_t, A_t) := Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$

Learning q_* (optimal policy)

Choose action A_{t+1} from S_{t+1} based on policy derived from Q

Behavior policy

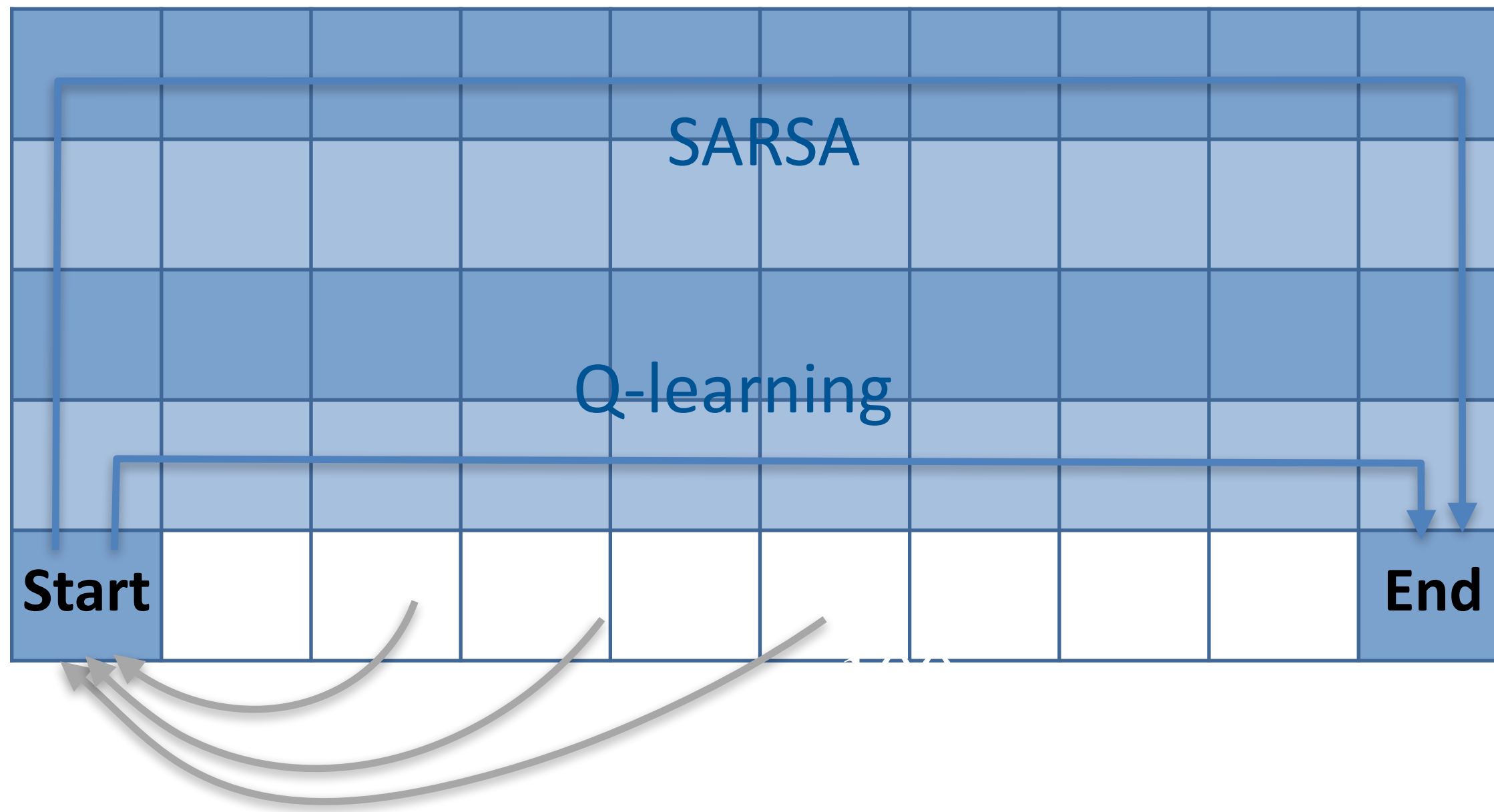
Until $S_t = S_T$ (terminal state of the episode)

Converges towards optimality much faster (provided values of all state-action pairs continue to be updated)

Discussion

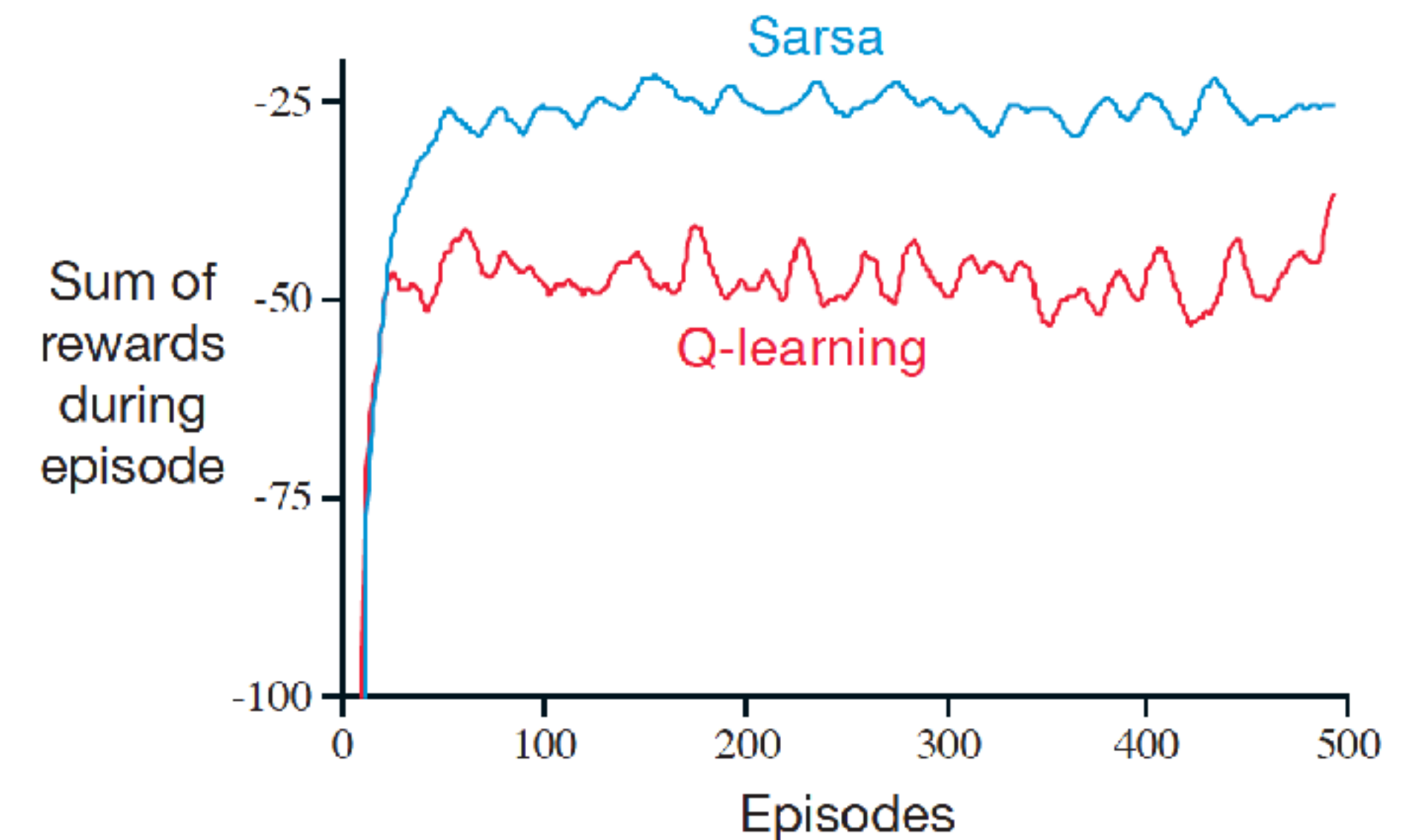
- Q-learning directly learns the optimal policy
- SARSA learns a near-optimal policy by exploring
 - Need to use decaying ε if we want to learn the optimal policy (because it is on policy)
- Q-learning (off - policy) has a higher per sample variance

Example: cliff walking



- Want to go from Start to End
- Rewards: -1 for all steps (time taken), but
- Penalty for falling off the cliff is high (-100)
- Need to start afresh as well

- Q-learning learns values for the optimal policy (along the edge)
 - But occasionally falls off the cliff (ϵ -greedy)
- SARSA learns the policy it follows
 - Learns a safer policy



Discussion

- Q-learning directly learns the optimal policy
- SARSA learns a near-optimal policy by exploring
 - Need to use decaying ε if we want to learn the optimal policy (because it is on policy)
- Q-learning (off - policy) has a higher per sample variance
- SARSA is more conservative
 - Exploratory moves may have penalties
 - SARSA would avoid a dangerous optimal path
 - Q-learning would not bother
- Real life
 - Simulation mode, or low cost fast iterating environment: Q-learning
 - Real life mode, where mistakes can be costly: SARSA

Expected SARSA

Q-Learning update step: $Q(S_t, A_t) := Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$

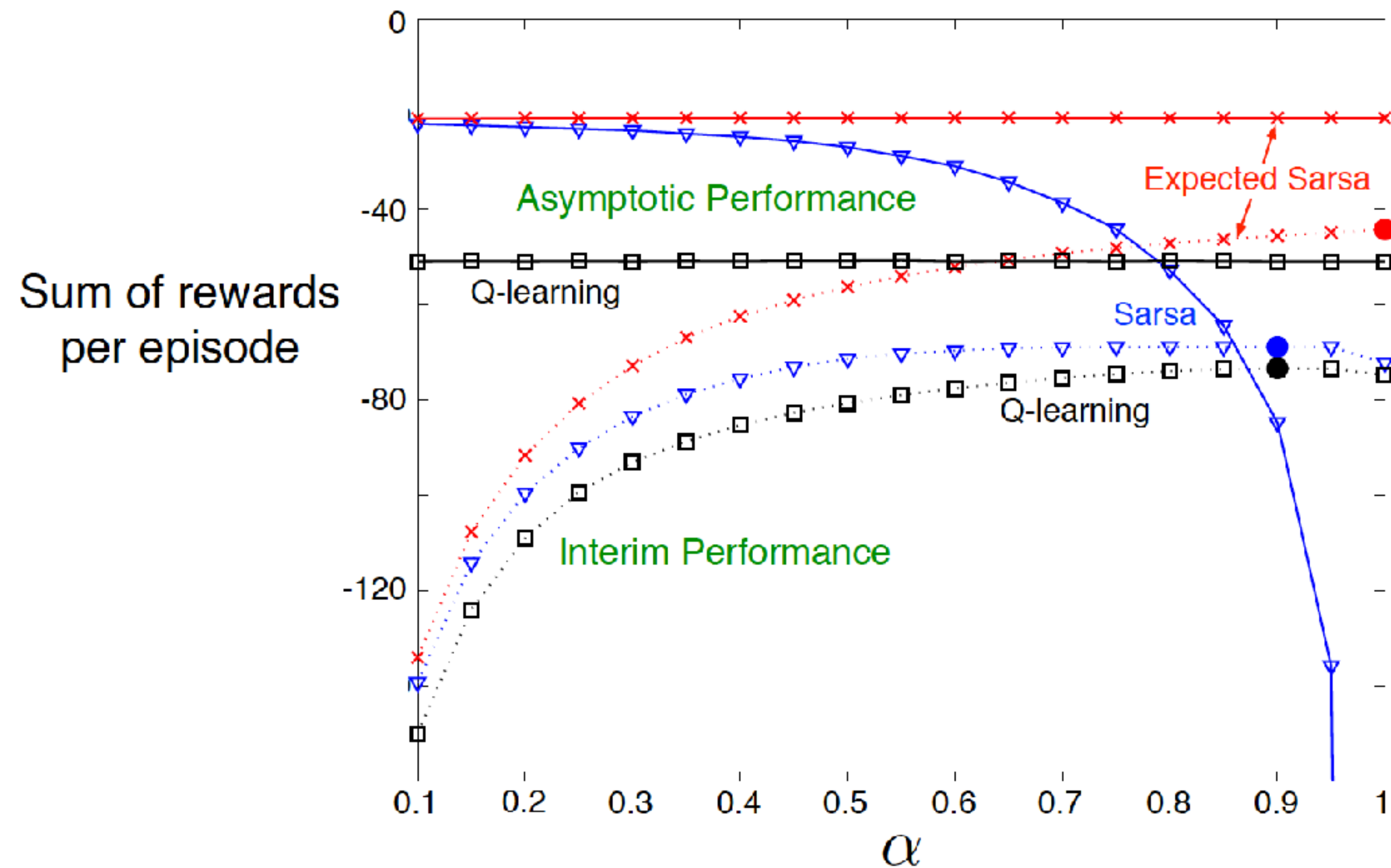
Replace the maximum by expected value

Expected SARSA update step: $Q(S_t, A_t) := Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \mathbb{E}_\pi [Q(S_{t+1}, A_{t+1}) | S_{t+1}] - Q(S_t, A_t) \right]$

or: $Q(S_t, A_t) := Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \sum_a \pi(a | S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t) \right]$

Expected SARSA: deterministically moves in the direction SARSA is expected to move.

Expected SARSA: Performance



- Expected SARSA is computationally more expensive
- With same amount of experience, performs better than SARSA
- Example: performance on the cliff walking problem
 - Asymptotic: average over 10,000 episodes
 - Interim: average over the first 100 episodes

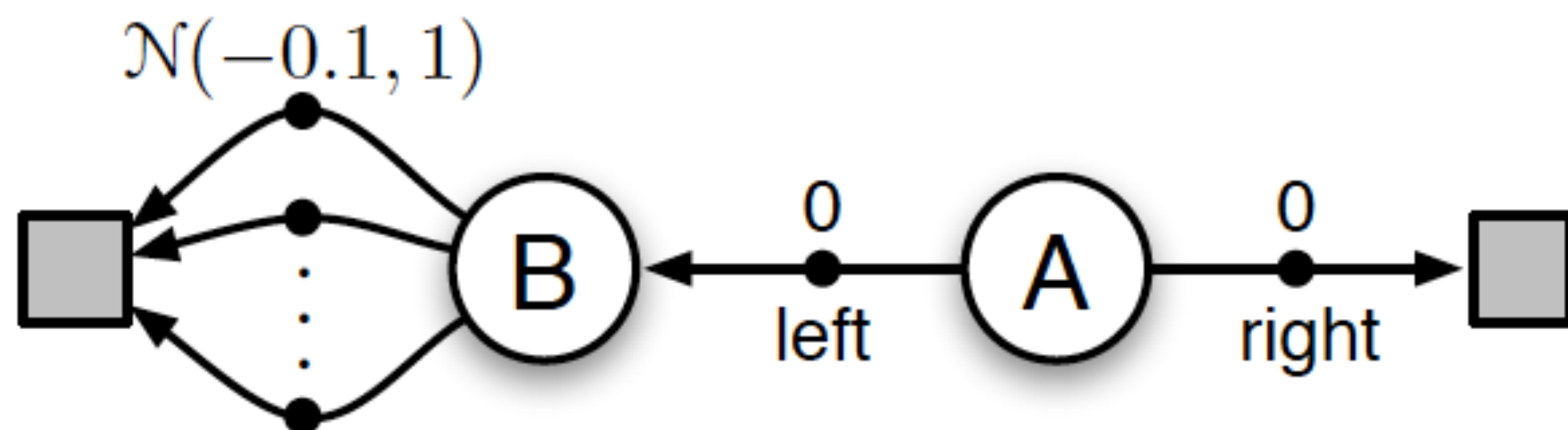
Maximization Bias

- Policy control algorithms involve “maximization” in construction of the target policy

$$\text{Q-Learning: } Q(S_t, A_t) := Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

- Target $Y = R_{t+1} + \gamma \max_a Q(S_{t+1}, a)$
- We are taking samples of going from S_t to S_{t+1} by action A_t , updating $Q(S_t, A_t)$ whenever we do so.
- Ideally, our target should be $Y = R_{t+1} + \gamma \max_a \mathbb{E} [Q(S_{t+1}, a)]$
- But the expectation of the target is $\mathbb{E} [Y] = R_{t+1} + \gamma \mathbb{E} \left[\max_a Q(S_{t+1}, a) \right]$
- During the updates
 - $Q(S_{t+1}, a)$ is an estimate, so $\max_a Q(S_{t+1}, a)$ is the **maximum of estimated values**
 - We actually want an **estimator for the maximum (true) value** $\max_a q(S_t, a)$
- This induces a bias, called the **maximization bias**
- Intuitive idea: **max** is a convex function, so $\max(\mathbb{E}(X)) \leq \mathbb{E}(\max(X))$ [Jensen’s inequality]

Maximization Bias: Example



- Two terminal states
- Always start at A, two actions from A: left and right
- Right action from A $\rightarrow$ reach terminal state with reward 0
- Left action from A $\rightarrow$ reach B.
- From B, the next left action can reach a terminal state, with reward $\sim \mathcal{N}(-0.1, 1)$
- State value of B = -0.1 , $q(A, \text{right}) = 0$.
- However, there would be positive rewards from B in sampled actions.
- So, Q-learning initially favours left action going to B.

Double Learning

- Idea: learn two independent estimators Q_1 and Q_2 .
- At any point of time:
 - use one for choosing the action: $A^* = \operatorname{argmax}_a Q_1(a)$
 - use the other for providing the estimate of its value: $Q_2(A^*) = Q_2(\operatorname{argmax}_a Q_1(a))$
- These would be unbiased estimators: $\mathbb{E} [Q_2(A^*)] = q(A^*)$.
- Reverse the role of the two estimators Q_1 and Q_2 to update the estimator Q_1
- How to incorporate this into policy learning (all algorithms for MDPs)?
- Double Q-learning (Q_1 and Q_2 to be interchanged randomly):

$$Q_1(S_t, A_t) := Q_1(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q_2 \left(S_{t+1}, \operatorname{argmax}_a Q_1(S_{t+1}, a) \right) - Q_1(S_t, A_t) \right]$$

- How to choose actual actions?
- Can combine the estimators Q_1 and Q_2 for that

Double Q-Learning

Initialize $Q_1(s, a), Q_2(s, a) \in \mathbb{R}$ arbitrarily for all s, a pairs; $q(S_T, a) = 0$ for terminal S_T and all a

Loop for each episode (an episode can be infinitely long as well)

Initialize start state S

For each step in episode

Choose action A from S based on policy derived from $Q_1 + Q_2$

Take action A , observe reward R and next state S'

if $\text{rand}(0,1) < 1/2$:

$$\text{Update } Q_1(S, A) := Q_1(S, A) + \alpha \left(R + \gamma Q_2(S', \arg\max_a Q_1(S', a)) - Q_1(S, A) \right)$$

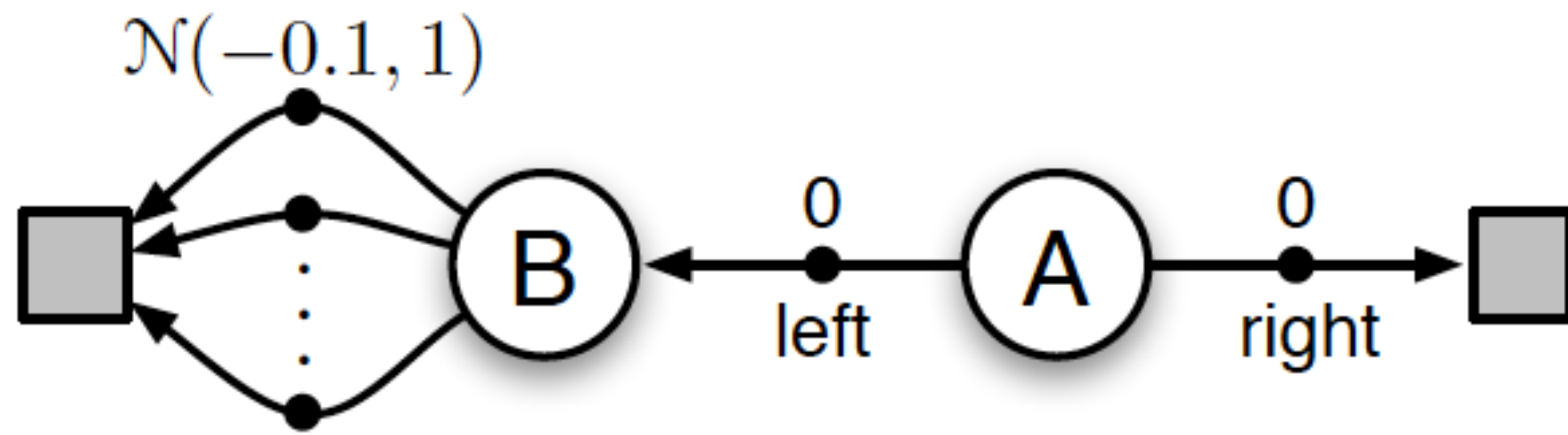
else:

$$\text{Update } Q_2(S, A) := Q_2(S, A) + \alpha \left(R + \gamma Q_1(S', \arg\max_a Q_2(S', a)) - Q_2(S, A) \right)$$

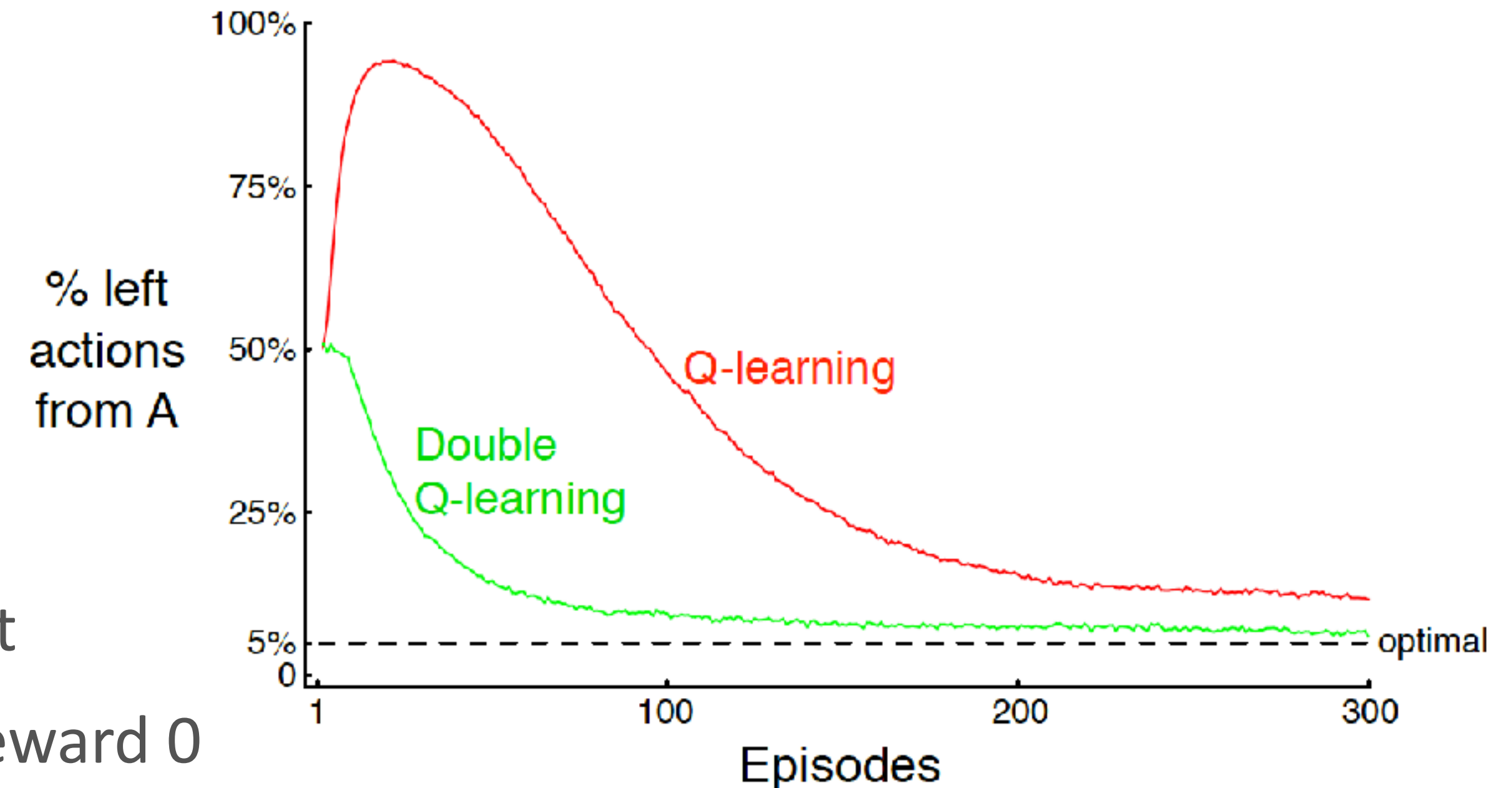
$$S := S'$$

Until $S_t = S_T$ (terminal state of the episode)

Double Q-Learning to mitigate Maximization Bias

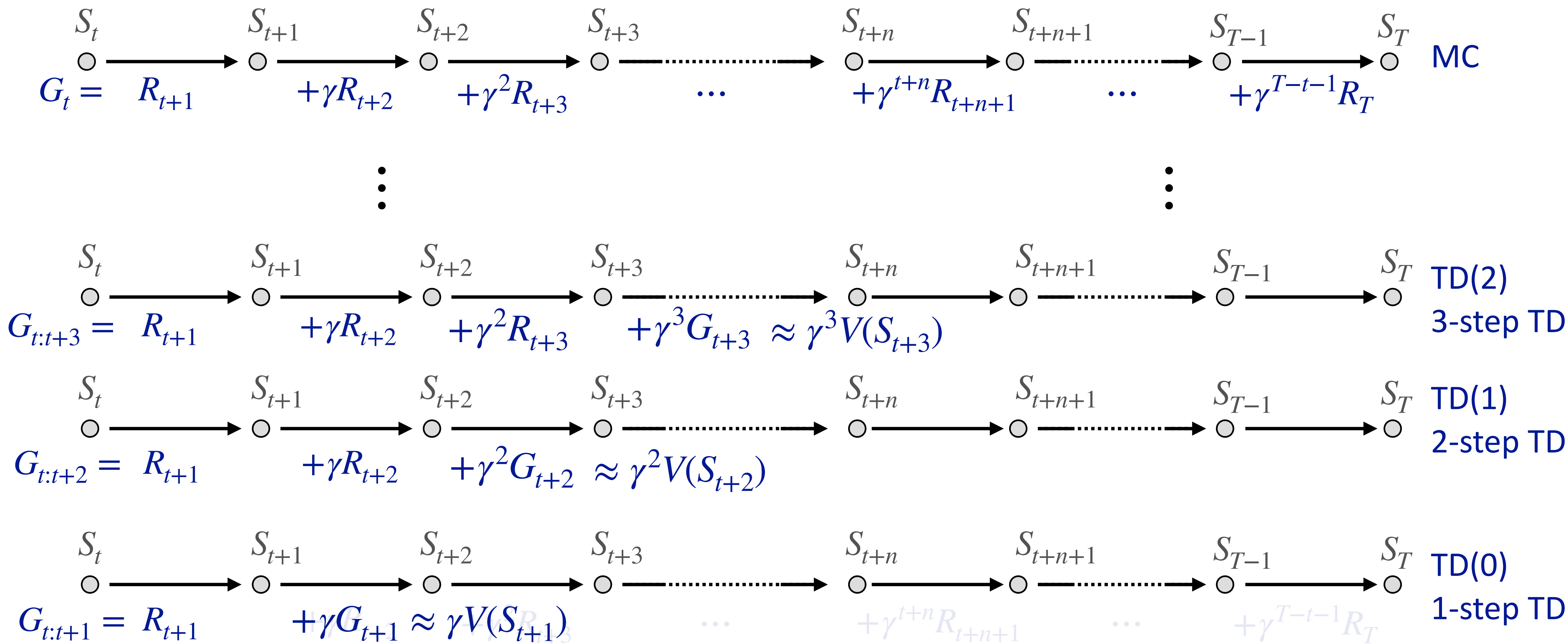


- Two terminal states
- Always start at A, two actions from A: left and right
- Right action from A $\rightarrow$ reach terminal state with reward 0
- Left action from A $\rightarrow$ reach B.
- From B, the next left action can reach a terminal state, with reward $\sim \mathcal{N}(-0.1, 1)$
- State value of B = -0.1 , state value of A = 0.
- However, there would be positive rewards from B in sampled actions.
- So, Q-learning initially favours left action going to B.



n -Step Bootstrapping

Generalization of MC and TD(0)



n -Step TD

- The target: $G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V_{t+n-1}(S_{t+n})$
- Note: $G_{t:t+n}$ is not available at time-step t , need to run through till time-step $t + n$, observe reward R_{t+n} and compute $V_{t+n-1}(S_{t+n})$.
- The update for n -step policy evaluation: $V_{t+n}(S_t) = V_{t+n-1}(S_t) + \alpha [G_{t:t+n} - V_{t+n-1}(S_t)]$.
- No update is possible during the first $n - 1$ steps of an episode.
- To make up for that, an equal number of updates are made at the end of the episode.
- The target for n -step policy control (in terms of action-values):

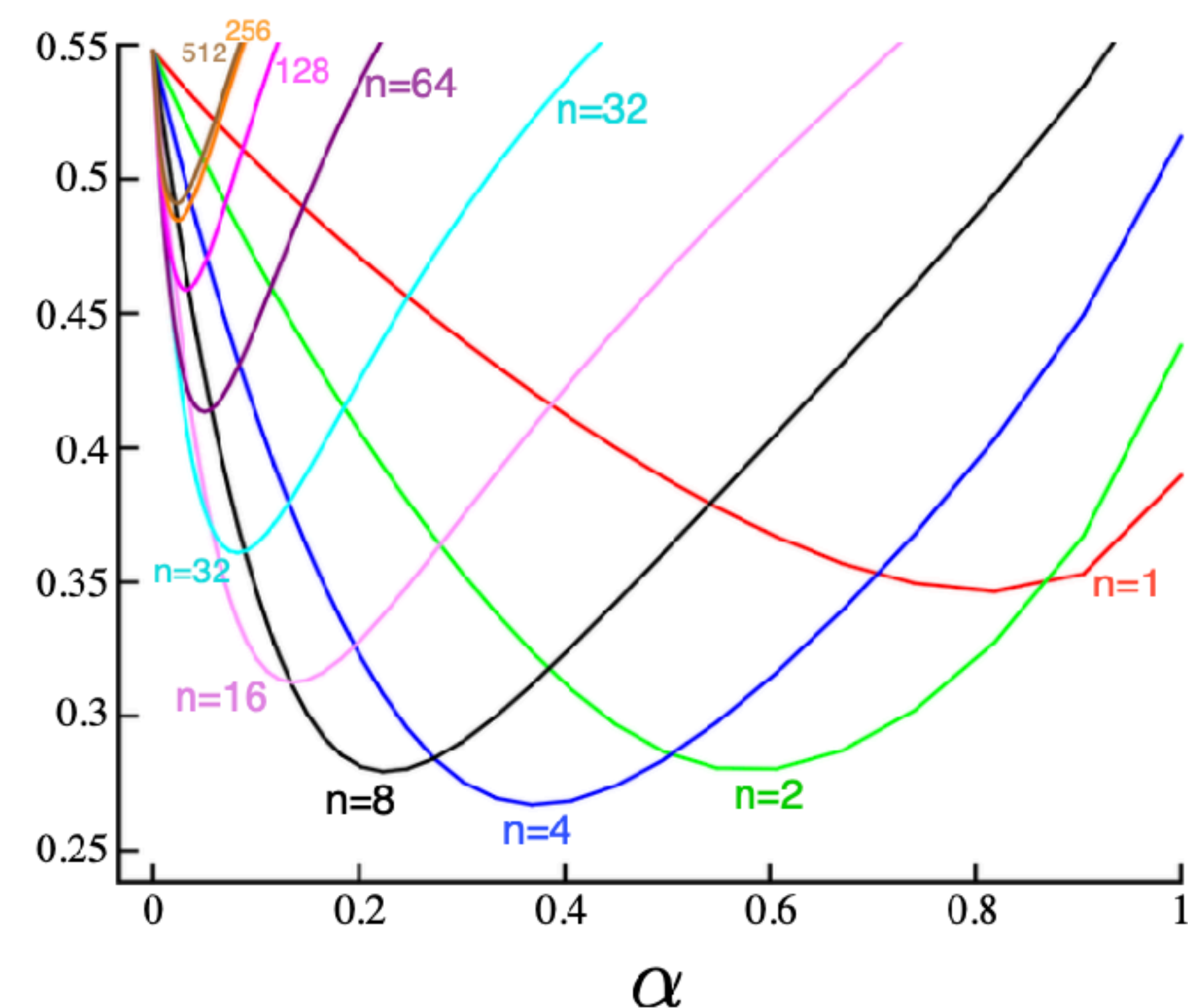
$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n Q_{t+n-1}(S_{t+n}, A_{t+n})$$
- Update for n -step policy control: $Q_{t+n}(S_t, A_t) = Q_{t+n-1}(S_t, A_t) + \alpha [G_{t:t+n} - Q_{t+n-1}(S_t, A_t)]$.

Varying n for n -Step TD



- Markov Reward Process (MRP): similar to MDP, but without the choice of an action
 - Useful for value prediction only (assume actions are random among the permissible ones)
- Example: states 0 to 6
 - States 0 and 6 are terminal states
 - An episode starts at 3 (middle)
 - Actions: random (with equal probability) towards left or right
 - Rewards: 1 for the right terminal state, zero for all other states
 - An example episode: 3, 4, 3, 4, 3, 2, 1, 2, 1, 0

Average
RMS error
over 19 states
and first 10
episodes



n -Step SARSA (on policy TD control)

Initialize $Q(s, a)$ arbitrarily for all $s \in \mathcal{S}, a \in \mathcal{A}$. Initialize π as an ε -greedy policy w.r.t. Q , or a fixed given policy

Loop for each episode:

Initialize start state $S_0 \neq$ terminal, an action $A_0 \sim \pi(\cdot | S_0)$, $T = \infty$.

For $t = 0, 1, 2, \dots$:

If $t < T$:

Take action A_t , observe and store R_{t+1} and S_{t+1}

If S_{t+1} is terminal, set $T = t + 1$; else select and store action $A_{t+1} \sim \pi(\cdot | S_{t+1})$.

$\tau = t - n + 1$ (we will update the estimates for time τ)

If $\tau \geq 0$:

$$G = \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$$

If $\tau + n < T$, then $G = G + \gamma^n Q(S_{\tau+n}, A_{\tau+n})$.

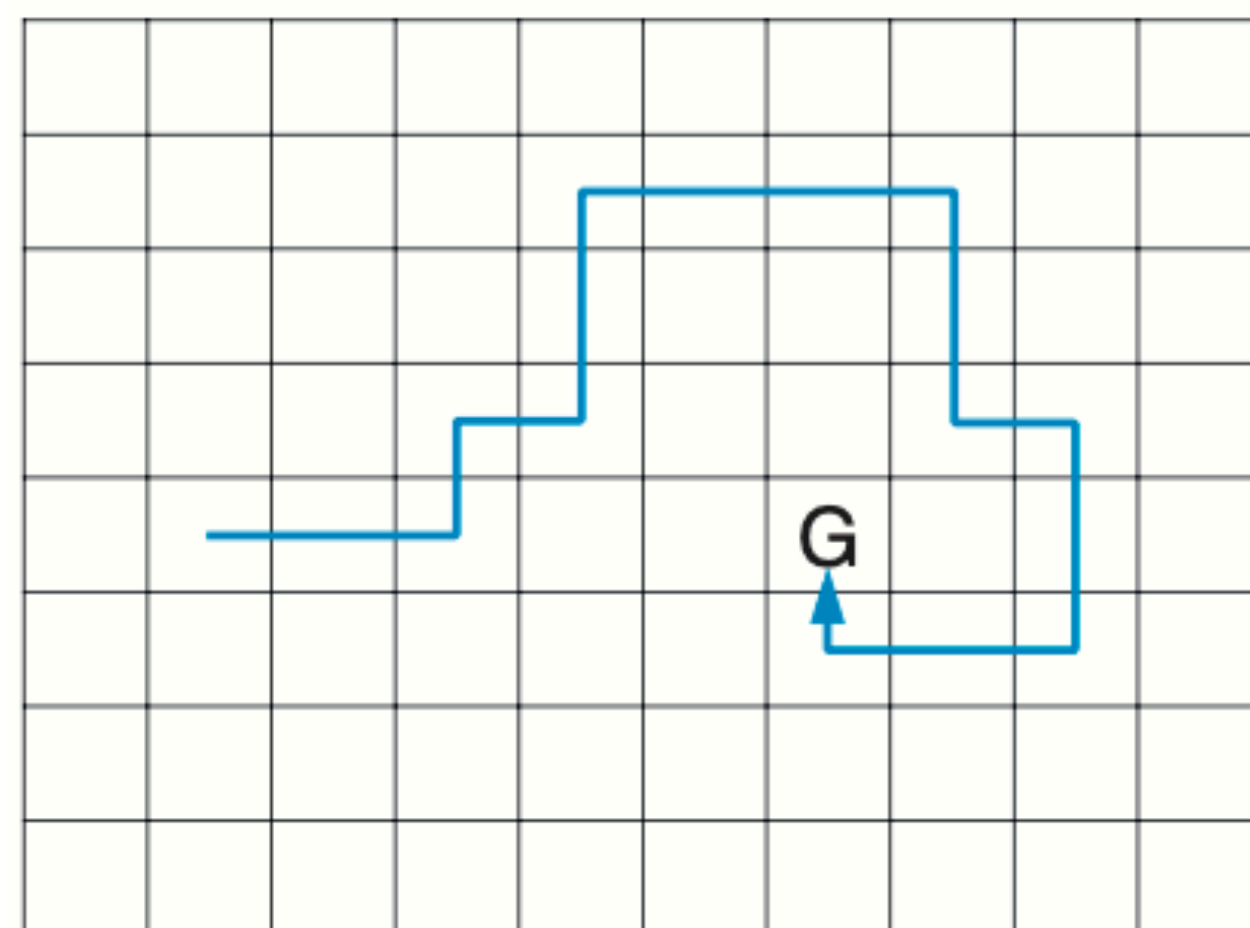
$Q(S_\tau, A_\tau) = Q(S_\tau, A_\tau) + \alpha [G - Q(S_\tau, A_\tau)]$. Keep $\pi(\cdot | S_\tau)$ ε -greedy w.r.t. Q

Until $\tau = T - 1$

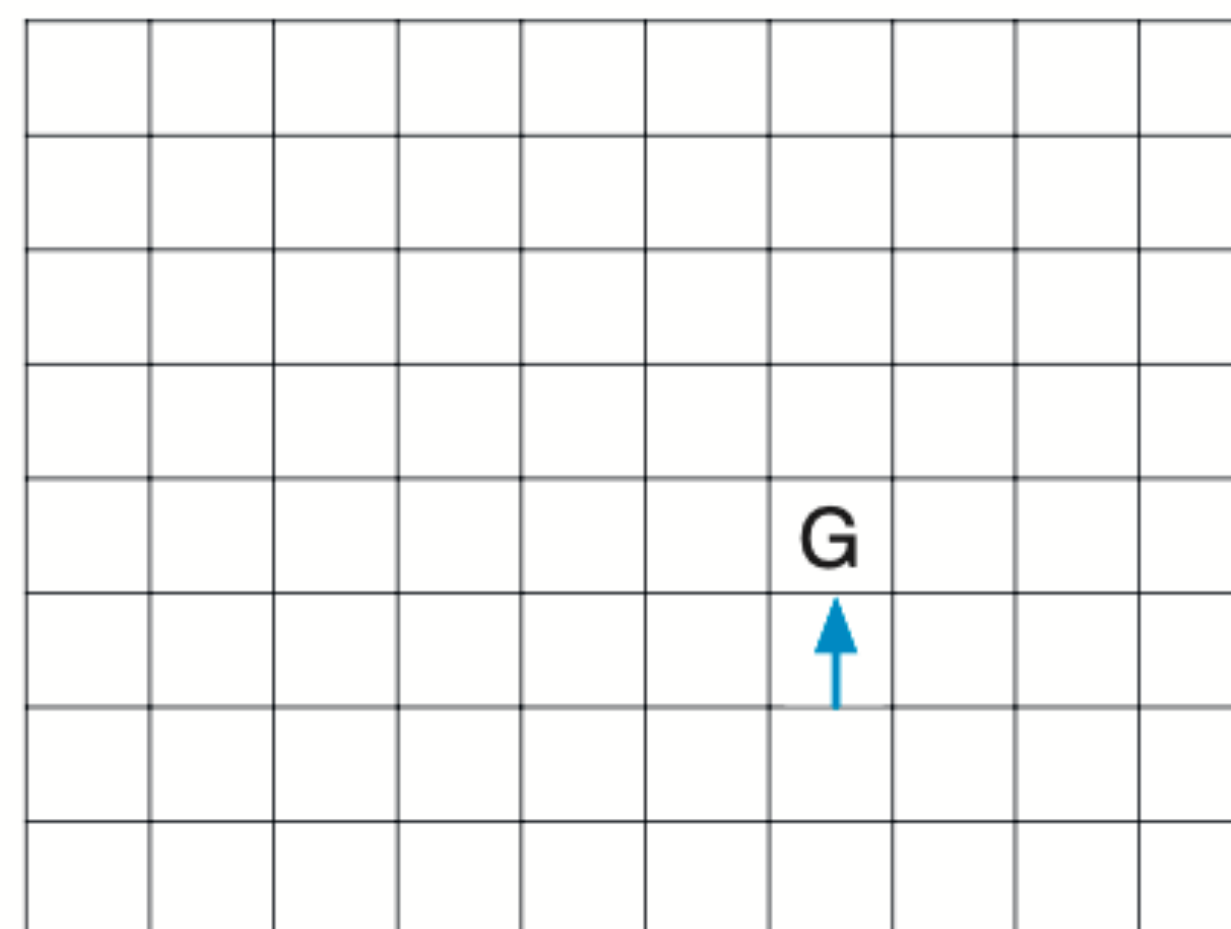
Example

- A gridworld where reward for all states is zero except the goal(s) where the reward is high (+ve)

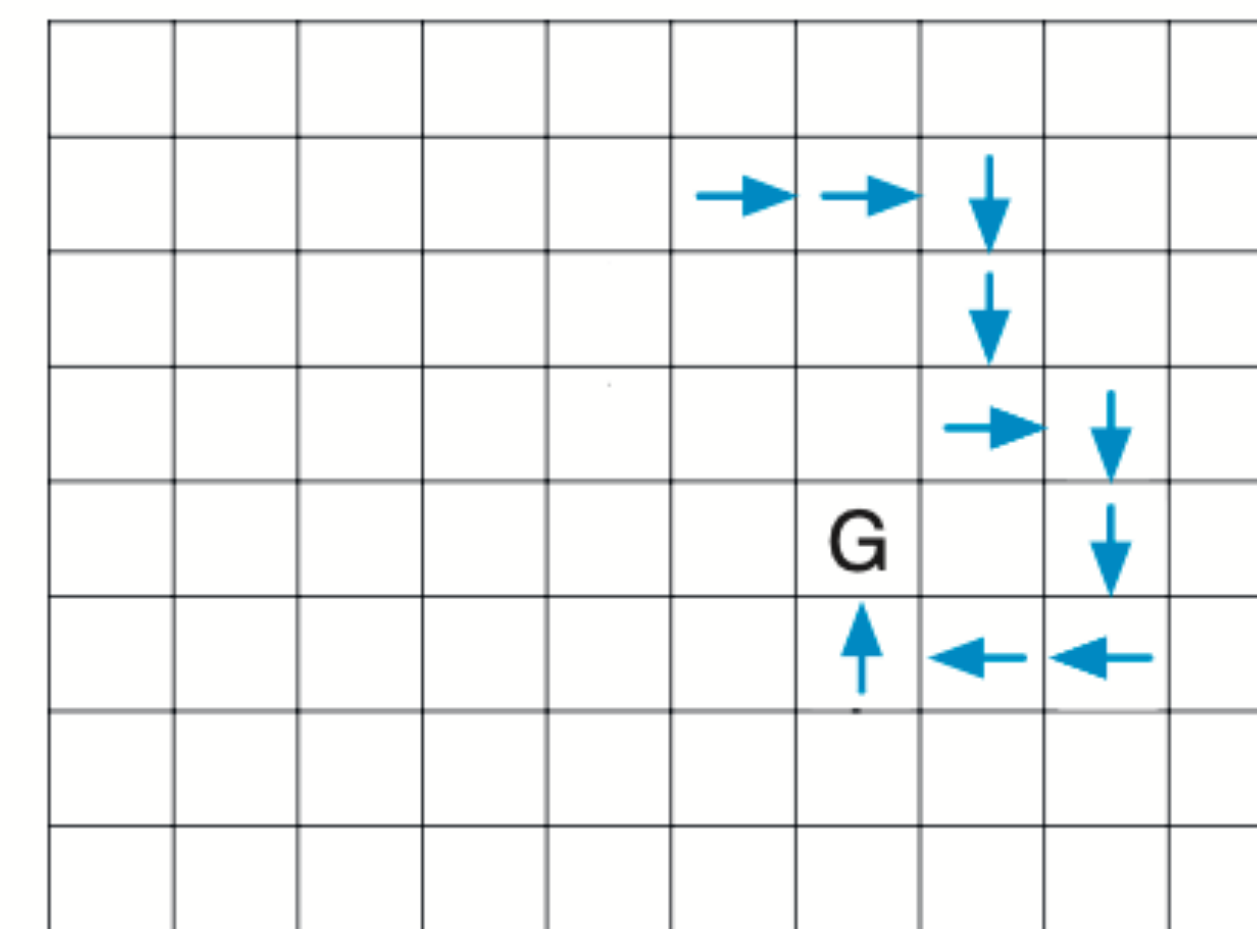
Path taken



Action values increased
by one-step Sarsa



Action values increased
by 10-step Sarsa



- Question: in such cases, how would MC work?
 - better than TD, or worse?

Choice of n for n -step TD algorithms?

- Small n : the update relies heavily on recent rewards
 - learning process becomes more step-wise reactive and stable but potentially slower in converging
- Larger n makes the updates incorporate more future rewards
 - makes the learning faster and more forward-looking, but potentially unstable
- Choosing Fixed n : A common approach is to start with small values of n (like 1 or 2) for stability and then increase it if faster learning is needed or if the problem is more deterministic
- Adaptive n
 - Error Reduction: n can be increased if the error in the value estimates is large, allowing more future rewards to be considered, and decreased when the error is small
 - Variance Reduction: If the learning process becomes too unstable (high variance), n can be reduced

Choice of n for n -step TD algorithms?

- Contextual or Domain-Based n
 - Problem-Specific: Depending on the specific problem, a suitable n can be chosen
 - For example, in environments where rewards are sparse and delayed, a larger n might be more appropriate to capture the effect of future rewards.
- Empirical Tuning: n can be tuned based on empirical results
 - Running multiple experiments with different values of n and selecting the one that offers the best performance (e.g., highest cumulative reward, fastest convergence) is a practical approach.
- Combining Multiple n Values
 - Instead of committing to a fixed n , take a weighted combination
 - We will see this in $TD(\lambda)$ later in this course

References

- Sutton, R.S. and Barto, A.G., 2018. Reinforcement learning: An introduction. MIT press.
<http://incompleteideas.net/book/the-book.html> (primary reference), Chapter 6